



SDEV2171

Lesson 06: Navigation Fundamentals

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, students should be able to:

- Explain the purpose of navigation in a mobile application
- Configure basic multi-screen navigation using Expo Router
- Define and register application screens within a navigation structure
- Navigate between screens using navigation actions
- Debug common navigation configuration and runtime issues

Lesson Roadmap

1. Reconnect lesson-05 structure to screen flow
2. Introduce routes, layouts, and stack navigation
3. Analyze a simple two-screen flow
4. Guided navigation build
5. Status check and troubleshooting
6. Checkpoint and lesson-07 handoff



ESSENTIAL
TO ALBERTA



Starting Point From Lesson 05

- use the same Expo project from lesson 05
- confirm your styled screen still runs
- identify two content areas that can become separate screens

If your project is not ready

- if your lesson-05 app runs, keep using your own project
- if your project is broken, restore the last working lesson-05 state first
- if you cannot recover in time for class progress, use the instructor reference app and record your next step

Why Navigation Matters

- real apps rarely keep everything on one screen
- navigation defines how users move between tasks
- screen flow affects clarity, focus, and app structure
- the clearer your lesson-05 sections are, the easier this split becomes



ESSENTIAL
TO ALBERTA



Course Navigation Standard

- this course uses Expo Router on the SDK 56 course scaffold
- Expo Router is the recommended navigation approach for Expo projects
- route files become screens
- a stack layout controls how screens are shown and moved

Translation

- a route file is a screen
- `src/app/_layout.tsx` defines the stack
- navigation actions move between those screens

Rough Flow Versus Clear Flow

One crowded screen

- everything competes for attention
- there is no clear path between tasks
- later interactions become harder to organize

Two-screen flow

- the home screen introduces the main content
- the detail screen focuses on one task or section
- navigation makes the app structure visible

The Three Core Files

File	Job
<code>src/app/_layout.tsx</code>	defines the stack layout
<code>src/app/index.js</code>	first screen in the flow
<code>src/app/details.js</code>	second screen in the flow

Mental model

- file names create the route structure
- the stack layout wraps those routes into a navigation flow

Minimal Layout Pattern

```
import { Stack } from 'expo-router';

export default function Layout() {
  return <Stack />;
}
```

What this means

- the app now has stack-based navigation
- route files inside `src/app/` become stack screens

Minimal Navigation Action Pattern

```
import { useRouter } from 'expo-router';
import { Pressable, Text } from 'react-native';

export default function HomeScreen() {
  const router = useRouter();

  return (
    <Pressable onPress={() => router.push('/details')}>
      <Text>Open details</Text>
    </Pressable>
  );
}
```

Rule for today

- use one forward action and one back action
- keep the route names simple and visible

Stack Mental Model

- `router.push('/details')` adds the details screen on top of the current screen
- `router.back()` returns to the previous screen in that stack history
- the current screen changes because the top of the stack changes

Plain-language version

- `push` moves forward to a new screen
- `back` returns to the screen underneath it

Guided Build Order

1. confirm the lesson-05 baseline
2. split the old `src/app/App.js` screen into route files
3. create the stack layout in `_layout.tsx`
4. create `index.js`
5. create `details.js`
6. connect the screens with one forward and one back action
7. test the full flow

Screen Flow Target

- `index.js` acts as the home screen
- `details.js` acts as the second screen
- one button or press action moves forward
- one action returns to the first screen
- both screens still show readable lesson-05 styling

Recovery Path If Navigation Is Blocked

- `configured` : both screens open and the flow works
- `partial` : route files exist, but one action or screen still fails
- `blocked` : the project cannot complete the navigation flow during class

If you are blocked

- use the instructor-guided reference path instead of starting a new setup attempt
- inspect `_layout.tsx` , `index.js` , and `details.js`
- trace the navigation flow in code and on the instructor demo
- record the failing step and one next action before class ends

Important fallback rule

- blocked students do not need to install and run the fallback app themselves during class
- the fallback path is to inspect and understand a known-good route structure

How Screens Are Registered Here

- in this course scaffold, route files become screens automatically
- `<Stack />` is enough to create the basic stack flow
- `Stack.Screen` is optional in this lesson and is used only for simple screen titles or options

Translation

- file creation plus the stack layout handles the screen registration
- stack options handle naming or header changes

Common Navigation Mistakes

- `_layout.tsx` does not return `Stack`
- a route file is named incorrectly
- the pushed path does not match the route file
- the app still points at an older single-screen file
- one screen works, but there is no return path

Quick Route Reasoning Example

If this file exists

- `src/app/details.js`

Then this path should work

- `router.push('/details')`

Debug rule

- check the file name first
- then check the pushed path
- then check that `_layout.tsx` returns `Stack`
- then check whether the old `App.js` content was split into the route files you expect

Guided Experimentation

- change one screen title in the stack options
- move one content block from the home screen to the detail screen
- test the forward action twice
- test the back action twice
- keep the cleanest working flow

Navigation Status Check

Before the checkpoint, record one state

- configured
- partial
- blocked

Every student should also record

- the current failing step, if any
- the next action to try before lesson 07

Checkpoint

Students should be ready to show

- a two-screen app with working navigation, or
- a partial or blocked state with documented progress and next action

Students should also be able to answer

- Which file defines the stack layout?
- Which file becomes the first screen?
- Which navigation action moves to the second screen?
- How are screens registered in this lesson structure?
- What did you check first when navigation failed?

What We Learned Today

- why single-screen apps often need navigation
- how Expo Router uses files as routes
- how `_layout.tsx` defines a stack flow
- how to move between screens with simple router actions
- how to classify progress as configured, partial, or blocked

Next Lesson Bridge

- keep this same navigation project
- lesson 07 adds user-triggered interaction and input handling
- next class uses the navigation flow you built today as the structure for those interactions

Ready for lesson 07 means

- full readiness: your status is `configured`
- if your status is `partial` or `blocked`, you must leave with a documented remediation plan before lesson 07
- at minimum, you should know how to rebuild `_layout.tsx`, `index.js`, and `details.js`